

Lesson 1 — REST API Fundamentals

Building a Precise Mental Model of Distributed Systems and Web Architectures

Before writing code or configuring ASP.NET Core, it is necessary to establish a precise mental model of distributed systems, web architectures, and the terminology used to describe them. Misunderstanding these foundational concepts often leads to architectural flaws that affect scalability and maintainability.

Core Definitions

What is an API?

An Application Programming Interface (API) is a formal contract between two distinct pieces of software. It defines the operations that can be performed, the required inputs, and the expected outputs. An API abstracts the underlying implementation; the consumer of the API does not need to know how the logic is executed, only how to communicate with the interface.

What is a Web API?

A Web API is an API exposed over a network using web technologies, primarily the Hypertext Transfer Protocol (HTTP). While an API can be internal to a single application (like a class library), a Web API specifically facilitates communication between distributed systems over a network boundary.

Client, Server, Request, and Response

Web APIs operate on a Client-Server architecture:

- **Client:** The entity (such as a browser, mobile application, or another server) initiating the communication.
- **Server:** The application hosting the API, processing the incoming data, and executing the business logic.
- **Request:** The structured HTTP message sent from the client to the server, detailing the desired operation.
- **Response:** The structured HTTP message sent back from the server to the client, containing the outcome of the operation and any requested data.

This separation of concerns allows the client and server to evolve independently, provided the API contract remains intact.

REST and RESTful Architecture

What is REST?

REST (Representational State Transfer) is an architectural style for distributed hypermedia systems, introduced by Roy Fielding in his 2000 doctoral dissertation.

It is critical to understand that REST is **not a protocol** and **not a standard**. It is a set of architectural constraints. HTTP is the protocol most commonly used to implement REST, but they are not synonymous.

REST vs. Web API vs. HTTP

- **HTTP:** The communication protocol (defined by RFC 9110) that dictates how messages are formatted and transmitted.
- **Web API:** Any programmatic interface exposed over HTTP.
- **RESTful API:** A Web API that strictly adheres to the constraints defined by the REST architectural style.

A Web API might use SOAP, GraphQL, gRPC, or ad-hoc RPC (Remote Procedure Call) patterns. A Web API is considered "RESTful" only when it follows REST constraints.

Key REST Constraints

1. Statelessness

In a RESTful architecture, communication must be stateless. This means every HTTP request from a client must contain all the information necessary for the server to understand and process it. The server must not rely on any stored context (such as an in-memory session state) from previous requests.

Why it matters: Statelessness directly enables horizontal scalability. If a server goes down, or if traffic spikes require load balancing across multiple servers, any server in the cluster can handle any request because no server needs to synchronize session state with the others.

2. Resources and Representations

REST revolves around the concept of **Resources**. A resource is any conceptual entity the API exposes, such as a **Vehicle**, an **Order**, or a **User**.

A **Representation** is the current state of that resource formatted in a specific way for transmission over the network. When a client requests a **Vehicle** resource, the server does not send the actual database row or the internal C# object; it sends a *representation* of it, typically serialized as JSON or XML.

Resource-Based vs. Action-Based Design

One of the most significant architectural shifts when adopting REST is moving from action-based design (RPC) to resource-based design.

The Action-Based (RPC) Approach

In an action-based design, URLs represent commands or verbs. The HTTP method is often ignored, with `GET` or `POST` used generically for all operations.

```
GET /api/getProducts
GET /api/deleteProduct?id=5
POST /api/createProduct
```

Technical problems with this approach:

1. **URI Proliferation:** Every new operation requires a completely new URL path, leading to bloated routing tables.
2. **Protocol Ignorance:** It ignores the semantic meaning built into the HTTP protocol. Using `GET` to delete data violates HTTP specifications. `GET` is defined as a safe, read-only operation. Network infrastructure (like proxies, caches, and web application firewalls) assumes `GET` requests do not modify data and may cache or pre-fetch them, which can result in accidental data deletion if `GET` is misused for destructive actions.

The RESTful (Resource-Based) Approach

In REST, URLs represent nouns (the resources), and the HTTP methods (`GET`, `POST`, `PUT`, `DELETE`) serve as the verbs.

```
GET    /api/products    (Retrieve a representation of the products collection)
POST   /api/products    (Append a new product to the collection)
DELETE /api/products/5   (Delete the product resource identified by ID 5)
```

Why this approach is preferable:

1. **Predictability (Uniform Interface):** The API contract becomes highly predictable. If a consumer knows the resource URL is `/api/vehicles`, they immediately know how to query, create, or delete based on standard HTTP methods.
2. **Proper HTTP Utilization:** It aligns with the RFC 9110 HTTP semantics specification. This allows infrastructure to correctly route, optimize, and cache requests. A proxy knows it can

safely cache a `GET /api/products` response, but knows it must invalidate that cache if it sees a `POST /api/products` request pass through.

Testing Understanding

Before we move on to Lesson 2 (HTTP Fundamentals), please review the following exercise and conceptual questions.

Practical Exercise

Assume you are building an API for an auto repair shop. You have a conceptual resource called a `RepairTask`. Write down the exact HTTP method and URL path you would design for the following four operations, strictly adhering to resource-based RESTful principles:

1. Retrieve a list of all repair tasks.
2. Retrieve the details of a specific repair task with the ID of 42.
3. Create a brand new repair task.
4. Delete the repair task with the ID of 42.

Conceptual Questions

1. Why does adhering to the "statelessness" constraint improve the ability to horizontally scale a Web API?
2. Explain the technical difference between a "Resource" and a "Representation" in the context of REST.
3. If a developer uses `GET /api/repairTasks/delete/42` to remove a record, what specific HTTP protocol rule does this violate, and what unexpected behavior might occur as a result of network infrastructure interacting with this endpoint?